

Student Name: Manula Manjitha Hindurangala
Student ID: 300390724
Progress Report Number: 2

Work Date/Hours Logs

Date	Number of Hours	Description of work done
Feb 28, 2026	7.5	Created baseline metric calculation module (baseline.py) with methods for collecting, calculating, and storing normal system metrics
		Added baseline metrics collection script (collect_baseline.py) for interactive baseline collection
		Implemented simple traffic generator for reliability testing
		Integrated baseline logic into backend application (app.py) and added /api/status endpoint
		Updated backend Dockerfile to create persistent data directory for baseline storage
		Updated backend dependencies in requirements.txt
Mar 1, 2026	6	Created anomaly_detector.py with full monitoring class - background thread, sensitivity thresholds, anomaly confirmation logic
		Added anomaly detector endpoints to Flask app: /api/detector/status, /api/detector/history, /api/detector/start, /api/detector/stop
		Created test script test_anomaly_detector_pod.py to run detector inside Kubernetes pod
		Fixed baseline module path in collect_baseline.py with permanent sys.path.append() solution
Mar 2, 2026	1	Created the progress report
Total	14.5	

Summary Description of Work Done During This Reporting Period

During this reporting period, I completed half of the **phase 2 of my research project**: the implementation of a complete anomaly detection system that continuously monitors metrics and triggers classification when anomalies are detected.

Work Completed:

1. **Baseline Calculator Enhancement:** Created and refined the `baseline.py` module which collects normal traffic patterns and establishes statistical baselines (mean, standard deviation) for key SRE metrics including request rate, error rate, latency percentiles, CPU usage, and memory usage.
2. **Anomaly Detector Implementation:** Developed the `anomaly_detector.py` class that runs as a background thread, querying Prometheus every 15 seconds, comparing current metrics against baselines using configurable sensitivity thresholds, and confirming anomalies after consecutive detections to reduce false positives.
3. **REST API Integration:** Added four new endpoints to the Flask backend:
 - `/api/detector/status` - Check detector running state and current anomalies
 - `/api/detector/history` - View historical anomaly records
 - `/api/detector/start` - Manually start the detector
 - `/api/detector/stop` - Manually stop the detector
4. **Testing & Validation:** Created a test script to run inside Kubernetes pods, successfully verifying end-to-end functionality. The system now actively tracks anomalies with 50+ consecutive detections and maintains history of 10+ anomaly events.

Issues Encountered:

- **Module Path Resolution:** The baseline collection script initially failed when run inside pods due to Python path issues. Resolved by adding permanent `sys.path.append('/app/src')` to the script.
- **Baseline Persistence:** After pod restarts, baseline files were lost. Implemented Docker volume configuration in the Dockerfile to persist data across restarts.
- **Network Connectivity:** Initial attempts to run collection scripts from host VM failed due to Prometheus service name resolution. Solved by running scripts inside the pod using `kubect exec`.

Next Steps:

- Implement the Classification Engine with research-specific logic
- Create rules to distinguish security incidents from operational incidents
- Identify attack types (DDoS, brute force, config error)
- Calculate severity based on user impact (SRE perspective)
- Integrate with anomaly detector callback

Repo Check-in of Implementation Completed

The following files/folders have been checked into the GitHub repository since the last progress report:

backend/src/baseline.py

- Core class for baseline calculation with methods for collecting metrics, calculating statistics, saving/loading baseline, and deviation detection

backend/src/anomaly_detector.py

- Complete anomaly detector implementation with background thread, sensitivity thresholds, anomaly confirmation logic, and history tracking

backend/src/app.py (updated)

- Added imports, detector initialization, and four new REST endpoints

scripts/collect_baseline.py (updated)

- Added permanent path fix with `sys.path.append('/app/src')`
- Improved error handling and user interface

scripts/test_anomaly_detector_pod.py

- New test script for running detector inside Kubernetes pod
- Includes callback demonstration and real-time status display

backend/Dockerfile (updated)

- Added `RUN mkdir -p /app/data && chown -R appuser:appuser /app/data`
- Ensures baseline persistence across pod restarts

backend/requirements.txt (updated)

- Verified all dependencies are properly listed

All code has been tested and verified working in the Kubernetes environment. The system successfully collects baselines, runs the anomaly detector continuously, and reports real-time anomalies via the REST API.

AI Use Section

AI Tool Name	Version, Type Account	Specific feature for which the AI tool was used	Value Addition (What value did you add over and above what AI did for you?)
DeepSeek	DeepSeek Chat, Free Account	Code structure for anomaly detector class and background threading	Adapted the threading implementation to work specifically with Kubernetes pod lifecycle and added custom sensitivity thresholds based on SRE research principles
DeepSeek	DeepSeek Chat, Free Account	PromQL query formulation for metric collection	Validated and tested each query against actual Prometheus data, adjusted time windows (5m) based on real system behavior
DeepSeek	DeepSeek Chat, Free Account	Error handling patterns for network timeouts	Customized retry logic and timeout values based on observed cluster performance
DeepSeek	DeepSeek Chat, Free Account	REST API endpoint design	Added specific error responses and status codes aligned with research requirements
DeepSeek	DeepSeek Chat, Free Account	Dockerfile configuration for data persistence	Verified the directory permissions and user contexts specifically for non-root appuser in our container

Appendix: Prompt History

Prompt 1 (Feb 28, 2026):

"Help me create a baseline calculator class in Python that collects metrics from Prometheus, calculates mean and standard deviation, and saves to JSON file. It needs to work inside a Kubernetes pod."

Prompt 2 (Feb 28, 2026):

"Write a PromQL query to get request rate, error rate, p95 latency, p99 latency, CPU usage, and memory usage from a Flask app with prometheus_client."

Prompt 3 (Feb 28, 2026):

"How to structure a background thread in Python that runs every 15 seconds without blocking the main Flask application?"

Prompt 4 (Mar 1, 2026):

"Design an anomaly detector class that compares current metrics with baseline, uses sensitivity thresholds, and requires 2 consecutive detections to confirm anomaly."

Prompt 5 (Mar 1, 2026):

"Write Flask REST endpoints for starting, stopping, and checking status of a background thread."

Prompt 6 (Mar 1, 2026):

"My Python script can't find a module when running inside a pod even though the file exists. How to fix import paths permanently?"

Prompt 7 (Mar 1, 2026):

"Create a test script that demonstrates the anomaly detector callback with real-time status updates."